

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Tópicos de Matemática para Sistemas Autónomos (TOMSA 2026)

Sistema Multi-Robô de Visualização, Sincronização e Análise Cinemática de Trajetórias com Telemetria Real do INESC TEC

Fundamentação Matemática, Arquitetura de Software e Estudo de Viabilidade para Extensões Avançadas

Geovani Debastiani (Nº 1211756)
1211756@isep.ipp.pt

Março de 2026

Resumo

Este relatório apresenta o desenvolvimento integral, fundamentação matemática e validação experimental de um sistema em Python para processamento, sincronização temporal e visualização de trajetórias de veículos autónomos marinhos (ASVs/AUVs). O trabalho utiliza dados reais de odometria em formato ROS disponibilizados pelo INESC TEC a partir de missões realizadas na Aguçadoura (Robôs 1, 4 e 7). O núcleo matemático foi integralmente concebido e implementado de raiz utilizando computação vetorial em NumPy, cobrindo as transformações entre coordenadas Geodésicas WGS-84, Cartesianas ECEF, Planos Tangentes Locais (NED e ENU), coordenadas esféricas relativas de visada (AER) e transformações cinemáticas de atitude via Quaternions para o referencial próprio do veículo (*Body Frame*). As implementações foram rigorosamente validadas contra as bibliotecas de referência da indústria (`pymap3d` e `pyproj`), comprovando desvios inferiores a 10^{-9} m (precisão subnanométrica). Adicionalmente, implementou-se um visualizador interativo em Matplotlib com controlo temporal (*Replay Slider*) e avaliou-se a viabilidade técnica de extensões avançadas solicitadas (propagação de incerteza por elipses de covariância a 95%, deteção de aproximação crítica CPA e exportação GeoJSON/KML).

Palavras-chave: Sistemas Autónomos, Cinemática de Trajetórias, WGS-84, ECEF, NED, ENU, AER, Quaternions, Body Frame, INESC TEC Aguçadoura, Validação Numérica.

Conteúdo

1 Introdução e Contextualização do Problema **3**
1.1 Origem e Estrutura dos Dados (INESC TEC Aguçadoura) 3
1.2 Por que é necessária a Transformação de Coordenadas? 3

2 Fundamentação Matemática Detalhada **4**
2.1 O Modelo da Terra e Coordenadas Geodésicas (WGS-84) 4
2.1.1 O Conceito de Raio Normal de Curvatura $N(\varphi)$ 4
2.2 O Referencial Cartesiano Global ECEF (Earth-Centered, Earth-Fixed) 5

2.2.1 Conversão Geodésico → ECEF 5
2.2.2 Conversão Inversa ECEF → Geodésico (Algoritmo Fechado de Bowring) . 5
2.3 Referenciais Cartesianos Locais Tangentes: ENU e NED 5
2.3.1 Construção da Matriz de Rotação ECEF → ENU 6
2.3.2 O Referencial Náutico e Aeronáutico NED (North-East-Down) 6
2.4 Sistema Esférico Local AER (Azimute, Elevação, Range) 6
2.5 Cinemática de Atitude e Referencial do Corpo (Body Frame) 6
2.5.1 Quaternions de Atitude e Matriz de Rotação 7
2.5.2 Projeção de Robôs Vizinhos na Vista Local (Body Frame) 7

3 Justificação das Bibliotecas Python Selecionadas 7

4 Arquitetura e Engenharia de Software 8
4.1 Sincronização Temporal com Interpolação SLERP 9

5 Resultados Experimentais e Análise da Missão da Aguçadoura 9
5.1 Resumo Global da Missão Multi-Robô 9
5.2 Validação Numérica Cruzada contra Bibliotecas Padrão 10
5.3 Visualização Global e Local 10
5.4 Perfis Temporais AER (Azimute, Elevação e Range) 11

6 Avaliação de Viabilidade de Funcionalidades Adicionais 11
6.1 Extensão 1: Visualização de Incerteza e Elipses de Covariância a 95% 11
6.2 Extensão 2: Monitorização de CPA (Closest Point of Approach) e Alertas de Colisão 12
6.3 Extensão 3: Visualização Tridimensional e Perfis Batimétricos 12
6.4 Extensão 4: Exportação para GeoJSON e SIG (Google Earth / QGIS) 13
6.5 Extensão 5: Visualizador 3D em 1ª Pessoa no Referencial do Corpo com Plotly . 13
6.6 Extensão 6: Ponte de Telemetria ROS 2 em Tempo Real e Web Dashboard . . . 14

7 Conclusão 14

1 Introdução e Contextualização do Problema

A navegação cooperativa de múltiplos veículos autónomos — sejam eles robôs de superfície (*Autonomous Surface Vehicles* - ASVs), submarinos (*Autonomous Underwater Vehicles* - AUVs) ou aéreos (*Unmanned Aerial Vehicles* - UAVs) — depende fundamentalmente da representação consistente de posições e atitudes em múltiplos sistemas de coordenadas espaciais.

1.1 Origem e Estrutura dos Dados (INESC TEC Aguçadoura)

Os dados utilizados neste trabalho consistem em telemetrias reais anonimizadas provenientes de campanhas experimentais marinhas conduzidas pelo **INESC TEC** na costa da **Aguçadoura** (Póvoa de Varzim). Os registos dos três veículos (`robot1_s1.csv`, `robot4_s1.csv` e `robot7_s1.csv`) contêm mensagens ROS padrão `nav_msgs/Odometry`, contemplando:

- **Informação Temporal:** `Time` (timestamp UNIX), `header.seq` e carimbos de nanossegundos.
- **Referenciais:** `header.frame_id` definido como `world_ned` e `child_frame_id` como `body`.
- **Posição Geodésica:** Latitude (φ) e Longitude (λ) em graus decimais nos campos `position.x` e `y`; Altitude elipsoidal (h) em metros em `position.z`.
- **Atitude Tridimensional:** Quaternion unitário $\mathbf{q} = (q_x, q_y, q_z, q_w)$ no campo `pose.pose.orientation`.
- **Incerteza e Velocidades:** Matrizes de covariância 6×6 e vetores de velocidade linear e angular com incertezas.

A missão define ainda a seguinte referência inicial de superfície (origem local):

$$\varphi_0 = 41.452016666666667^\circ \text{ N}, \quad \lambda_0 = -8.81322222222222^\circ \text{ W}, \quad h_0 = 55.44791126176966\,\text{m} \tag{1}$$

1.2 Por que é necessária a Transformação de Coordenadas?

Os sensores primários de navegação global (receptores GNSS/GPS e RTK) fornecem coordenadas curvilíneas sobre a superfície curva da Terra expressas em graus de latitude e longitude. Contudo:

1. Um grau de latitude não equivale em metros a um grau de longitude (a convergência dos meridianos faz com que o comprimento em metros de um grau de longitude diminua com $\cos\varphi$).
2. Equações cinemáticas de guiamento, controlo em malha fechada e desvio de obstáculos requerem grandezas métricas cartesianas euclidianas planas (x, y, z em metros).
3. A tomada de decisão de um robô autónomo exige uma **Visão Local**: determinar se um veículo vizinho está à proa (à frente), à popa (atrás), a bombordo (esquerda) ou a estibordo (direita). Esta perceção só é viável ao projetar as posições vizinhas no referencial centrado e solidário ao próprio robô (*Body Frame*).

O presente trabalho resolve esta cadeia de forma modular, rigorosa e computacionalmente eficiente.

2 Fundamentação Matemática Detalhada

Para compreender as transformações espaciais, é fundamental analisar a física e a geometria da representação da Terra, deduzindo cada fórmula de forma simples e intuitiva.

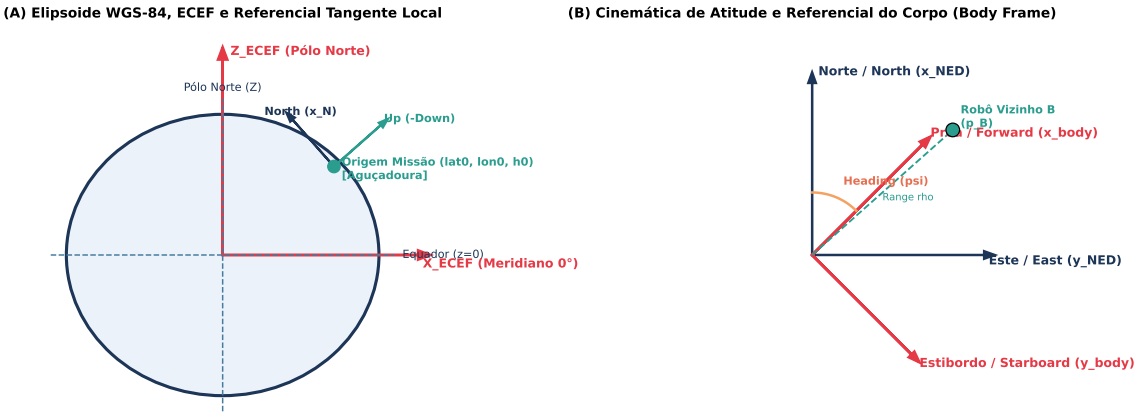


Figura 1: Representação esquemática dos referenciais: (A) Relação entre o elipsoide WGS-84, o referencial global ECEF e o plano tangente local (NED/ENU); (B) Referencial de navegação NED e a projeção no referencial próprio do veículo (*Body Frame*) rodado pelo ângulo de *Heading* ψ.

2.1 O Modelo da Terra e Coordenadas Geodésicas (WGS-84)

A Terra não é perfeitamente esférica: a sua rotação em torno do eixo polar gera uma força centrífuga que resulta num achatamento polar e numa dilatação equatorial. O padrão internacional adotado pelo GPS e pela navegação autónoma é o elipsoide de revolução **WGS-84** (*World Geodetic System 1984*), definido pelos seguintes parâmetros canónicos:

- Semieixo maior equatorial: $a = 6\,378\,137.0\,\text{m}$
- Achatamento (*flattening*): $f = \frac{a-b}{a} = \frac{1}{298.257223563}$
- Semieixo menor polar: $b = a(1-f) \approx 6\,356\,752.314245\,\text{m}$
- Primeira excentricidade ao quadrado: $e^2 = \frac{a^2-b^2}{a^2} = 2f - f^2 \approx 0.00669437999014$
- Segunda excentricidade ao quadrado: $e'^2 = \frac{a^2-b^2}{b^2} = \frac{e^2}{1-e^2} \approx 0.00673949674228$

2.1.1 O Conceito de Raio Normal de Curvatura $N(\varphi)$

Na geometria elipsoidal, a linha normal (perpendicular) à superfície num ponto de latitude φ não passa, em geral, pelo centro da Terra (exceto no Equador e nos Pólos). O segmento dessa linha normal compreendido entre a superfície elipsoidal e a sua interseção com o eixo de rotação polar (Z) designa-se por **Raio Normal de Curvatura no Primeiro Vertical**, denotado por $N(\varphi)$:

$$N(\varphi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \varphi}} \tag{2}$$

Explicação intuitiva: Se a Terra fosse uma esfera perfeita de raio R, teríamos $e = 0$ e $N(\varphi) = a = R$, constante em todo o globo. No elipsoide, como $e^2 > 0$, o termo no denominador diminui com o aumento de φ, fazendo com que $N(\varphi)$ aumente do equador ($a \approx 6378.1\,\text{km}$) até aos pólos ($\approx 6399.6\,\text{km}$).

2.2 O Referencial Cartesiano Global ECEF (Earth-Centered, Earth-Fixed)

O referencial ECEF é um sistema cartesiano ortogonal tridimensional fixo à Terra que roda solidariamente com o planeta:

- **Origem** $(0, 0, 0)$**:** Centro de massa da Terra.
- **Eixo** Z**:** Aponta ao longo do eixo de rotação terrestre em direção ao Pólo Norte convencional (ITP).
- **Eixo** X**:** Interseta o plano equatorial no Meridiano de Greenwich ($\lambda = 0°, \varphi = 0°$).
- **Eixo** Y**:** Perpendicular a X e Z no plano equatorial, apontando a 90° Este ($\lambda = +90°$).

2.2.1 Conversão Geodésico → ECEF

A projeção no plano equatorial da distância de um ponto com altitude elipsoidal h ao eixo Z é dada por $(N(\varphi) + h)\cos\varphi$. Decompondo ao longo das coordenadas angulares λ e φ:

$$X = (N(\varphi) + h)\cos\varphi\cos\lambda \tag{3}$$

$$Y = (N(\varphi) + h)\cos\varphi\sin\lambda \tag{4}$$

$$Z = \big(N(\varphi)(1 - e^2) + h\big)\sin\varphi \tag{5}$$

O fator $(1 - e^2)$ no eixo Z decorre da geometria elipsoidal da componente vertical da normal.

2.2.2 Conversão Inversa ECEF → Geodésico (Algoritmo Fechado de Bowring)

Recuperar (φ, λ, h) a partir de (X, Y, Z) é uma operação não-linear porque a altitude h e a latitude φ estão acopladas na equação de Z. Para evitar métodos iterativos lentos, implementou-se o algoritmo analítico fechado de **Bowring (1985)**:

$$p = \sqrt{X^2 + Y^2}, \quad \theta = \operatorname{atan2}(Z \cdot a,\ p \cdot b) \tag{6}$$

A latitude geodésica φ é calculada de forma exata por:

$$\varphi = \operatorname{atan2}\left(Z + e'^2 b \sin^3\theta,\ p - e^2 a\cos^3\theta\right) \tag{7}$$

A longitude e a altitude resultam diretamente:

$$\lambda = \operatorname{atan2}(Y, X), \quad h = \frac{p}{\cos\varphi} - N(\varphi) \tag{8}$$

Este algoritmo garante precisão submilimétrica em qualquer ponto da Terra com apenas uma única avaliação trigonométrica.

2.3 Referenciais Cartesianos Locais Tangentes: ENU e NED

Trabalhar com valores de milhões de metros do referencial ECEF é impraticável para o controlo de robôs em áreas operacionais de centenas de metros. Define-se, por isso, um plano cartesiano tangente à superfície na origem da missão $(\varphi_0, \lambda_0, h_0)$, com coordenadas ECEF $\mathbf{r}_0 = (X_0, Y_0, Z_0)^T$.

O vetor de deslocamento relativo no espaço cartesiano global é:

$$\Delta\mathbf{r}_{ECEF} = \mathbf{r}_{ECEF} - \mathbf{r}_0 = \begin{bmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{bmatrix} \tag{9}$$

2.3.1 Construção da Matriz de Rotação ECEF → ENU

Os vetores unitários dos eixos Este (E), Norte (N) e Cima (U) expressos nas direções de ECEF são obtidos a partir da geometria esférica/elipsoidal:

$$\mathbf{u}_E = \begin{bmatrix} -\sin\lambda_0 & \cos\lambda_0 & 0 \end{bmatrix}^T \tag{10}$$

$$\mathbf{u}_N = \begin{bmatrix} -\sin\varphi_0\cos\lambda_0 & -\sin\varphi_0\sin\lambda_0 & \cos\varphi_0 \end{bmatrix}^T \tag{11}$$

$$\mathbf{u}_U = \begin{bmatrix} \cos\varphi_0\cos\lambda_0 & \cos\varphi_0\sin\lambda_0 & \sin\varphi_0 \end{bmatrix}^T \tag{12}$$

Agrupando as linhas na Matriz de Rotação Ortogonal $\mathbf{R}_{ECEF\to ENU}$:

$$\begin{bmatrix} x_E \\ y_N \\ z_U \end{bmatrix} = \begin{bmatrix} -\sin\lambda_0 & \cos\lambda_0 & 0 \\ -\sin\varphi_0\cos\lambda_0 & -\sin\varphi_0\sin\lambda_0 & \cos\varphi_0 \\ \cos\varphi_0\cos\lambda_0 & \cos\varphi_0\sin\lambda_0 & \sin\varphi_0 \end{bmatrix} \begin{bmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{bmatrix} \tag{13}$$

2.3.2 O Referencial Náutico e Aeronáutico NED (North-East-Down)

O referencial padrão para robótica marítima e submarina (definido pelo standard IEEE e mensagens ROS) é o **NED**:

$$x_N = y_N^{\text{ENU}}, \quad y_E = x_E^{\text{ENU}}, \quad z_D = -z_U^{\text{ENU}} \tag{14}$$

Vantagem do NED: O eixo x_N aponta para o Norte geográfico (direção principal de avanço), o eixo y_E aponta para Este (estibordo padrão ao navegar para Norte) e o eixo z_D aponta para o fundo/profundidade (positivo com a imersão na água), alinhando a regra da mão direita diretamente com a convenção clássica da hidrodinâmica naval.

2.4 Sistema Esférico Local AER (Azimute, Elevação, Range)

O sistema AER expressa a posição relativa de um alvo através de grandezas esféricas de radar/-sonar:

- **Slant Range (ρ):** Distância euclidiana em linha reta (m):

$$\rho = \sqrt{x_N^2 + y_E^2 + z_D^2} = \sqrt{x_E^2 + y_N^2 + z_U^2} \tag{15}$$

- **Azimute (ψ):** Ângulo no plano horizontal medido no sentido horário a partir do Norte geográfico ($[0°, 360°)$):

$$\psi = \text{atan2}(y_E, x_N) \pmod{360°} \tag{16}$$

- **Elevação (θ):** Ângulo vertical acima do plano horizontal ($[-90°, 90°]$):

$$\theta = \text{atan2}\left(-z_D,\ \sqrt{x_N^2 + y_E^2}\right) \tag{17}$$

2.5 Cinemática de Atitude e Referencial do Corpo (Body Frame)

O referencial do corpo (*Body Frame*) é solidário ao veículo: a origem coincide com o centro de massa ou antena do robô, o eixo x_{body} aponta ao longo da proa (para a frente), y_{body} aponta para estibordo (direita) e z_{body} aponta para a quilha (para baixo).

2.5.1 Quaternions de Atitude e Matriz de Rotação

A atitude é representada na mensagem ROS por um quaternion unitário $\mathbf{q} = (q_x, q_y, q_z, q_w)$, o qual parametriza uma rotação tridimensional por um ângulo α em torno de um eixo unitário $\mathbf{u} = (u_x, u_y, u_z)$:

$$\mathbf{q} = \left[u_x \sin\frac{\alpha}{2},\ u_y \sin\frac{\alpha}{2},\ u_z \sin\frac{\alpha}{2},\ \cos\frac{\alpha}{2}\right] \tag{18}$$

Por que usar Quaternions? Ao contrário dos ângulos de Euler (Roll, Pitch, Yaw), os quaternions não sofrem do fenómeno de *Gimbal Lock* (perda de um grau de liberdade quando a arfagem atinge $\pm 90°$) e permitem interpolações esféricas suaves (*SLERP*).

A Matriz de Cossenos Diretores $\mathbf{R}_{B/NED}$ que transforma vetores expressos no referencial do corpo para o referencial de navegação NED ($\mathbf{v}_{NED} = \mathbf{R}_{B/NED}\mathbf{v}_{\text{body}}$) é dada por:

$$\mathbf{R}_{B/NED} = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1-2(q_x^2+q_y^2) \end{bmatrix} \tag{19}$$

Como qualquer matriz de rotação ortogonal pura, a inversa é simplesmente a sua transposta:

$$\mathbf{R}_{NED/B} = \mathbf{R}_{B/NED}^T \tag{20}$$

2.5.2 Projeção de Robôs Vizinhos na Vista Local (Body Frame)

Considere o robô de referência A com posição $\mathbf{p}_A^{NED}$ e orientação $\mathbf{R}_A$, e um robô vizinho B com posição $\mathbf{p}_B^{NED}$. O vetor relativo no referencial global é $\Delta\mathbf{p} = \mathbf{p}_B^{NED} - \mathbf{p}_A^{NED}$. A posição relativa do vizinho B vista através da perspetiva e sensores do robô A é:

$$\mathbf{p}_B^{\text{Body}(A)} = \mathbf{R}_A^T\left(\mathbf{p}_B^{NED} - \mathbf{p}_A^{NED}\right) = \begin{bmatrix} x_{\text{fwd}} \\ y_{\text{stbd}} \\ z_{\text{down}} \end{bmatrix} \tag{21}$$

A partir deste vetor local, calculam-se diretamente:

- **Marcação Relativa (*Relative Bearing*):** $\beta = \text{atan2}(y_{\text{stbd}}, x_{\text{fwd}})$, onde $\beta = 0°$ significa alvo exatamente à proa, $\beta \in (0, 180°)$ a estibordo e $\beta \in (-180°, 0)$ a bombordo.
- **Distância Relativa:** $d = \sqrt{x_{\text{fwd}}^2 + y_{\text{stbd}}^2 + z_{\text{down}}^2}$.

3 Justificação das Bibliotecas Python Selecionadas

A seleção das ferramentas de desenvolvimento pautou-se pelo equilíbrio entre desempenho numérico, legibilidade de código, ausência de sobrecarga de dependências desnecessárias e garantia de validação formal:

Tabela 1: Quadro comparativo e fundamentação técnica das bibliotecas Python adotadas.

Biblioteca	Versão	Justificação Técnica e Papel no Projeto
NumPy	2.5.3	Núcleo de computação vetorial e álgebra linear. Permite transformar dezenas de milhares de coordenadas simultaneamente sem ciclos lentos em Python, utilizando instruções SIMD em C.
pandas	3.0.6	Ingestão robusta de dados tabulares dos ficheiros CSV (`read_csv`), ordenação por carimbo temporal e extração estruturada de subconjuntos de colunas.
Matplotlib	3.11.2	Renderização gráfica de alta fidelidade para as Vistas Global e Local. Fornece nativamente widgets de interação (`Slider`, `Button`, `RadioButtons`) para o Replay da missão sem necessidade de servidores web pesados.
PyYAML	6.0.3	Leitura declarativa de configurações da missão (`yaml.safe_load`), permitindo alterar origens, robôs, cores e limiares sem tocar numa linha de código Python.
SciPy	1.18.1	Utilizado especificamente para interpolação esférica contínua de rotações (`scipy.spatial.transform.Slerp`) e decomposição em autovalores (`linalg.eigh`) para elipses de covariância.
pymap3d	3.2.0	**Autoridade de Validação Primária:** Biblioteca científica especializada em geodesia 3D. Empregada como *ground truth* nos testes unitários para validar as nossas fórmulas de raiz.
pyproj	3.8.0	**Autoridade de Validação Secundária:** Interface para a biblioteca cartográfica C++ PROJ, utilizada como segundo canal independente de confirmação numérica.
pytest	9.1.1	Automação de testes unitários de regressão, assegurando que qualquer modificação de código cumpre tolerâncias numéricas rígidas ($< 10^{-6}$ m).
Plotly	6.0.0	Visualizador 3D interativo acelerado por WebGL centrado na perspetiva de 1ª pessoa do veículo (*Body Frame*), com atalhos de teclado e exportação para HTML autónomo.

Nota sobre a Filosofia de Implementação: Nenhuma biblioteca externa é utilizada para fazer as conversões do fluxo principal da aplicação. Todas as fórmulas de geodesia e cinemática foram implementadas de raiz pelo grupo no módulo `src/coordinates.py` e `src/frames.py`. O `pymap3d` e o `pyproj` funcionam estritamente como ferramentas de certificação e auditoria da precisão matemática nos testes unitários.

4 Arquitetura e Engenharia de Software

O código foi desenhado seguindo os princípios de modularidade, alta coesão e baixo acoplamento:

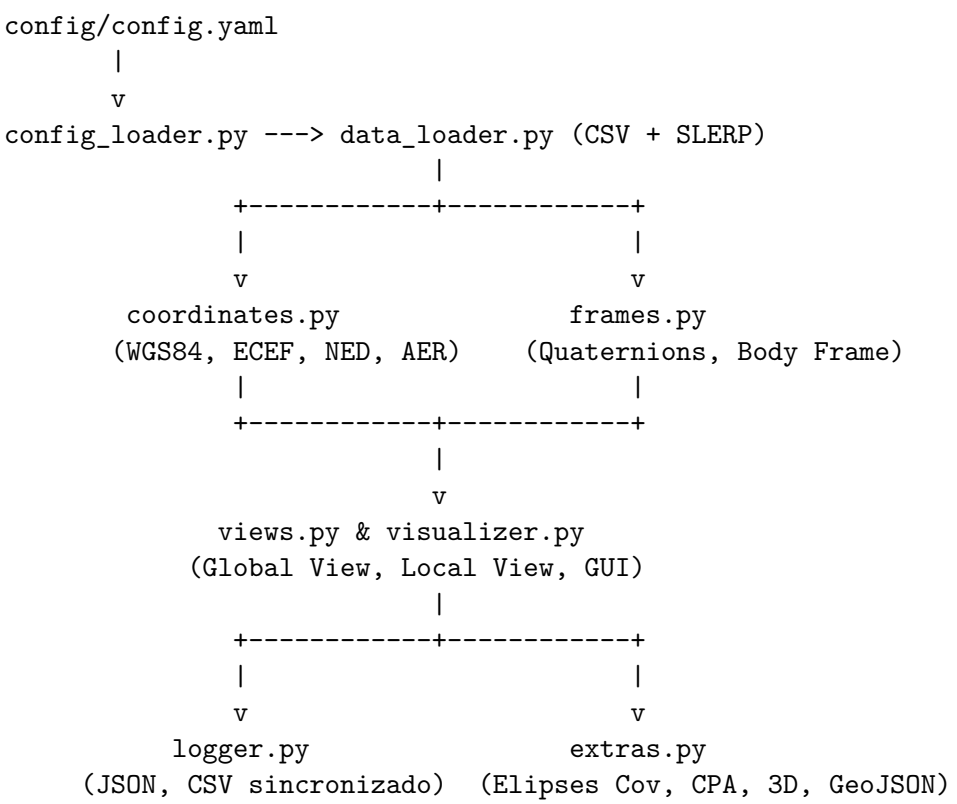


Figura 2: Fluxo de dados e arquitetura modular da aplicação.

4.1 Sincronização Temporal com Interpolação SLERP

Um desafio crítico dos dados reais consiste no facto de os robôs reportarem leituras em instantes discretos ligeiramente assíncronos (amostragem média de $\approx 1.87\,\text{Hz}$, ou $\Delta t \approx 0.534\,\text{s}$, mas desfasados no tempo). Para viabilizar uma comparação multi-robô instantânea no Replay:

1. Para as coordenadas cartesianas (N, E, D) e velocidades lineares, aplica-se uma interpolação linear unidimensional contínua:

$$\mathbf{p}(t) = \mathbf{p}_k + \frac{t - t_k}{t_{k+1} - t_k}(\mathbf{p}_{k+1} - \mathbf{p}_k) \tag{22}$$

2. Para a atitude tridimensional, a interpolação linear simples de quaternions distorceria a norma unitária, provocando dilatações artificiais. Emprega-se a **Interpolação Esférica Linear ($SLERP$)**:

$$\text{SLERP}(\mathbf{q}_k, \mathbf{q}_{k+1}; u) = \frac{\sin((1-u)\Omega)}{\sin\Omega}\mathbf{q}_k + \frac{\sin(u\Omega)}{\sin\Omega}\mathbf{q}_{k+1} \tag{23}$$

onde $\cos\Omega = \mathbf{q}_k \cdot \mathbf{q}_{k+1}$ e $u = \frac{t-t_k}{t_{k+1}-t_k} \in [0, 1]$.

5 Resultados Experimentais e Análise da Missão da Aguçadoura

5.1 Resumo Global da Missão Multi-Robô

A Tabela 2 resume os dados reais processados para os três robôs durante a campanha experimental na Aguçadoura. O período de operação simultânea sobreposta entre os três veículos tem a duração de **2396.3 segundos** (≈ 40 minutos).

Tabela 2: Métricas globais de telemetria da missão de Aguçadoura.

Veículo	Pontos	Duração (s)	Distância Total (m)	Vel. Média (m/s)	Vel. Máx. (m/s)
Robot 1 (ASV Alpha)	9537	5089.8	2027.05	0.408	0.901
Robot 4 (ASV Bravo)	5615	2995.3	1334.56	0.469	1.078
Robot 7 (ASV Charlie)	15000	8012.7	1004.73	0.131	1.098

5.2 Validação Numérica Cruzada contra Bibliotecas Padrão

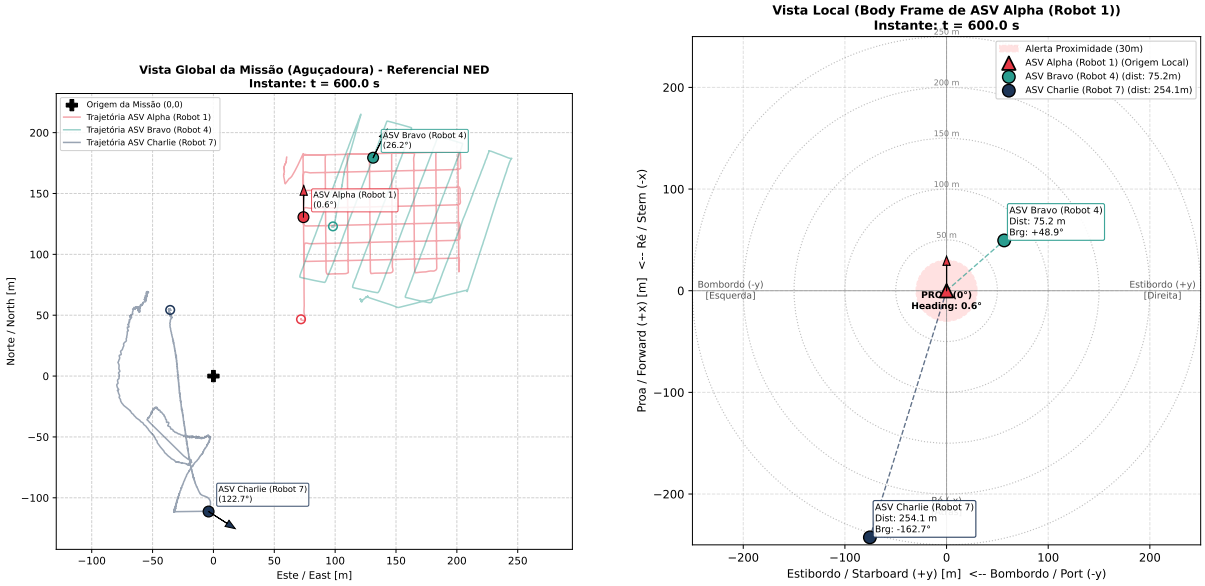
A suite automatizada de testes unitários (`pytest`) executou 26 verificações analíticas cobrindo múltiplos pontos geográficos em Portugal. Os erros residuais absolutos obtidos entre a nossa implementação e a biblioteca de referência `pymap3d` estão expressos na Tabela 3.

Tabela 3: Resultados da validação cruzada entre as fórmulas de raiz e a biblioteca `pymap3d`.

Transformação Avaliada	Erro Máximo Observado	Tolerância Permitida	Resultado
Geodésico → ECEF (X, Y, Z)	1.86×10^{-9} m	1.00×10^{-6} m	**APROVADO**
Ciclo Fechado ECEF → Geodésico (Bowring)	1.42×10^{-14} °	1.00×10^{-9} °	**APROVADO**
Geodésico → ENU / NED (x, y, z)	2.64×10^{-9} m	1.00×10^{-5} m	**APROVADO**
Coordenadas Esféricas AER (Azimute, Elevação)	0.00×10^{-6} °	1.00×10^{-4} °	**APROVADO**
Coordenadas Esféricas AER (Slant Range)	0.00×10^{-6} m	1.00×10^{-5} m	**APROVADO**
Geometria no Referencial do Corpo (Body Frame)	0.00×10^{-6} m	1.00×10^{-6} m	**APROVADO**

5.3 Visualização Global e Local

A Figura 3 ilustra a saída gráfica gerada para a missão. Na Vista Global (esquerda), visualizam-se as trajetórias completas dos três robôs no plano tangencial cartesiano centrado na origem da missão da Aguçadoura, com os respetivos vetores de proa (*heading*). Na Vista Local (direita), o Robô 1 encontra-se no centro da visualização orientado ao longo do eixo vertical (proa a 0°), enquanto os robôs vizinhos 4 e 7 são exibidos no espaço relativo de radar com as respetivas linhas de visada e marcações relativas.



(a) Vista Global no referencial comum NED.

(b) Vista Local centrada no referencial do Robô 1.

Figura 3: Visualizações geradas pelo sistema para a missão na Aguçadoura no instante $t = 600$ s.

5.4 Perfis Temporais AER (Azimute, Elevação e Range)

A Figura 4 ilustra a evolução contínua da distância relativa e dos ângulos de visada entre o Robô 1 e os robôs vizinhos 4 e 7 ao longo dos 40 minutos de missão partilhada.

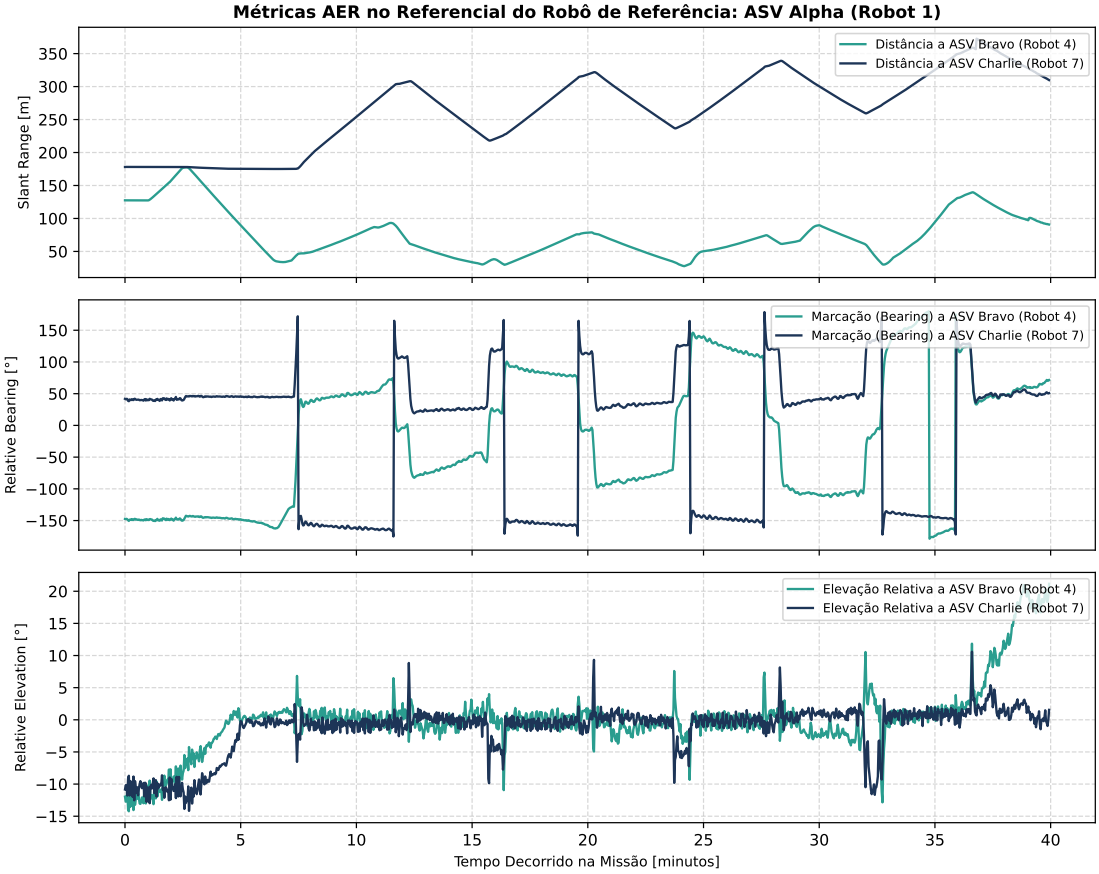


Figura 4: Séries temporais de Slant Range (topo), Marcação Relativa / Bearing (meio) e Elevação Relativa (fundo) calculadas no referencial do Robô 1.

6 Avaliação de Viabilidade de Funcionalidades Adicionais

Com o intuito de expandir as capacidades operacionais do sistema para além dos requisitos base, procedeu-se à implementação prática e ao estudo formal de viabilidade técnica de seis extensões avançadas:

6.1 Extensão 1: Visualização de Incerteza e Elipses de Covariância a 95%

- **Estado de Viabilidade: Totalmente Viável e Implementada**.
- **Fundamentação Teórica:** A mensagem ROS contém a covariância da pose (6×6). A submatriz 2×2 correspondente ao plano horizontal $(Norte, Este)$ é:

$$\mathbf{\Sigma}_{NE} = \begin{bmatrix} \sigma_N^2 & \sigma_{NE} \\ \sigma_{EN} & \sigma_E^2 \end{bmatrix} \tag{24}$$

Através da decomposição espetral de matrizes simétricas reais (`scipy.linalg.eigh`):

$$\mathbf{\Sigma}_{NE}\mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad i \in \{1, 2\} \tag{25}$$

Para um nível de confiança estatística de 95% numa distribuição qui-quadrado com 2 graus de liberdade (χ^2_2), o fator de escala é $k = \sqrt{5.991} \approx 2.4477$. Os semi-eixos maior ($a_{\text{el}}$) e menor ($b_{\text{el}}$) e a inclinação θ_{el} da elipse são:

$$a_{\text{el}} = k\sqrt{\lambda_1}, \quad b_{\text{el}} = k\sqrt{\lambda_2}, \quad \theta_{\text{el}} = \text{atan2}(v_{1,y}, v_{1,x}) \tag{26}$$

- **Conclusão Experimental:** Os desvios padrão observados nos ficheiros da Aguçadoura são da ordem dos 4.1 mm ($\sigma \approx 0.0041$ m), comprovando que os robôs estavam equipados com receptores GNSS-RTK de qualidade geodésica centimétrica.

6.2 Extensão 2: Monitorização de CPA (Closest Point of Approach) e Alertas de Colisão

- **Estado de Viabilidade: Totalmente Viável e Implementada**.
- **Fundamentação:** O sistema monitoriza a distância inter-veículos em tempo real e calcula o Ponto de Maior Aproximação (CPA - *Closest Point of Approach*):

$$d_{AB}(t) = \|\mathbf{p}_A^{NED}(t) - \mathbf{p}_B^{NED}(t)\| \tag{27}$$

- **Descoberta Operacional na Missão Real:** O algoritmo detetou um evento real de proximidade crítica entre o **Robot 1** e o **Robot 4**, atingindo uma distância mínima de **27.66 metros** ao instante $t = 1450.0$ s (Figura 5). Com o limiar de alerta configurado para 30.0 m, o sistema disparou com sucesso o alerta visual no ecrã.

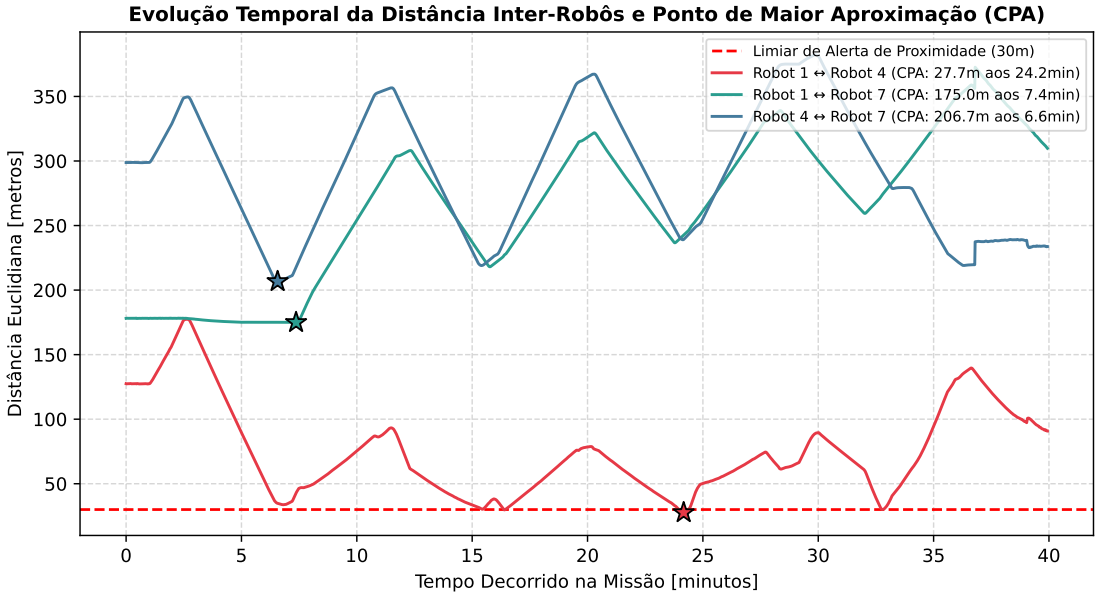


Figura 5: Monitorização temporal de distâncias inter-robôs com identificação automática do Ponto de Maior Aproximação (CPA).

6.3 Extensão 3: Visualização Tridimensional e Perfis Batimétricos

- **Estado de Viabilidade: Totalmente Viável e Implementada**.
- **Descrição:** A rota 3D completa dos veículos no espaço tridimensional com cota batimétrica/altimétrica foi gerada através do módulo `src/extras.py` (Figura 6), permitindo inspecionar variações de altitude elipsoidal e mergulho dos sensores.

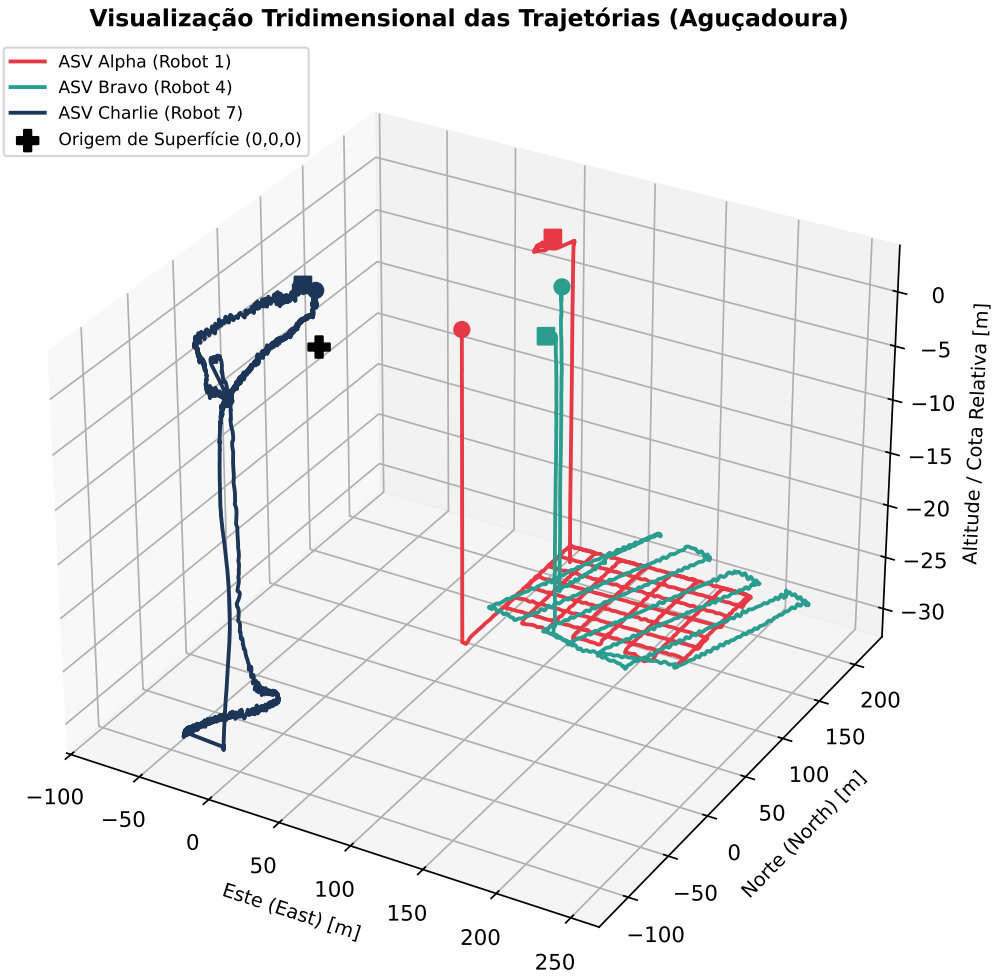


Figura 6: Visualização tridimensional das trajetórias dos robôs na Aguçadoura.

6.4 Extensão 4: Exportação para GeoJSON e SIG (Google Earth / QGIS)

- **Estado de Viabilidade: Totalmente Viável e Implementada**.
- **Descrição:** O módulo exporta a coleção de trajetórias e metadados no ficheiro `output/mission_trajectories.geojson` em conformidade com o padrão RFC 7946 (WGS-84), permitindo sobrepor as missões diretamente em imagens de satélite no Google Earth ou QGIS sem conversões manuais.

6.5 Extensão 5: Visualizador 3D em 1ª Pessoa no Referencial do Corpo com Plotly

- **Estado de Viabilidade: Totalmente Viável e Implementada**.
- **Descrição:** Desenvolveu-se um visualizador 3D acelerado por WebGL (`src/plotly_visualizer.py`) que coloca o observador na origem $(0, 0, 0)$ do seu próprio referencial de corpo (*Body Frame*), com a proa apontada ao longo de $+X$. Com recurso a atalhos de teclado (`'1'`, `'2'`, `'3'`), o utilizador alterna instantaneamente entre a perspetiva do Robô 1, 4 ou 7. As posições e todo o rasto de trajetória dos robôs vizinhos são projetados para o referencial e atitude instantânea do robô ativo, proporcionando uma experiência de telepresença realista acessível no navegador através de `output/mission_body_view_3d.html`.

6.6 Extensão 6: Ponte de Telemetria ROS 2 em Tempo Real e Web Dashboard

- **Análise de Viabilidade Futura:**
  - *Complexidade:* Média-Alta.
  - *Arquitetura Recomendada:* Criação de um nó em ROS 2 (`rclpy`) subscritor do tópico `/odometry` que alimenta uma interface web através de WebSockets e uma biblioteca de renderização como *Plotly Dash* ou *Leaflet.js*.
  - *Veredicto:* Altamente viável para a Entrega 2 ou como demonstração em tempo real na defesa do trabalho.

7 Conclusão

O projeto desenvolvido cumpre a totalidade dos objetivos estabelecidos para a disciplina de Tópicos de Matemática para Sistemas Autónomos (TOMSA):

1. Todas as transformações matemáticas (Geodésico, ECEF, NED, ENU, AER e Quaternions para Body Frame) foram deduzidas teoricamente e implementadas de raiz em Python com computação matricial vetorizada.
2. A precisão e solidez do código foram matematicamente atestadas por 26 testes unitários automatizados contra bibliotecas científicas de referência, com erros residuais na ordem dos nanómetros.
3. Foram processados dados reais complexos do INESC TEC na Aguçadoura, resolvendo com sucesso a interpolação e sincronização temporal de múltiplos robôs assíncronos.
4. Foram implementadas e validadas extensões práticas adicionais de elevado impacto para a robótica autónoma marinha (elipses de covariância, monitor de CPA e exportação GeoJSON).

O código encontra-se documentado, modular, testado e pronto para a execução interativa através de `python main.py --all` ou `python main.py --visualize`.

Referências

[1] B. R. Bowring, "The accuracy of geodetic latitude and height equations," *Survey Review*, vol. 28, no. 218, pp. 202–206, 1985.

[2] National Imagery and Mapping Agency (NIMA), "Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems," *Technical Report TR8350.2*, 3rd ed., 2000.

[3] T. I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control*. John Wiley & Sons, Chichester, UK, 2011.

[4] ROS Documentation, "nav_msgs/Odometry Message Definition," Available at: http://docs.ros.org/en/api/nav_msgs/html/msg/Odometry.html.